

Track the Cash Backend Handoff

Backend API contract for frontend implementation

Version 1.0 | September 2026

Purpose

Track the Cash predicts which ATM locations are most likely to be used for cybercrime cash withdrawal in the next 24 hours. It combines cybercrime complaints, mule-account geography, complaint velocity, historical risk, and a machine-learning risk model.

The frontend is an operational dashboard for two users:

- LEA field officer: find high-risk ATMs, inspect alert context, and prioritize field response.
- I4C administrator: monitor national trends, inspect cross-state networks, trigger simulations, notify teams, and export intelligence.

This document describes the backend contract the frontend should consume.

Local API

- Base URL: <http://localhost:8000>
- Swagger documentation: <http://localhost:8000/docs>
- Health check: [GET /health](#)
- API JSON schema: [GET /openapi.json](#)

Most business resources use [/api/v1](#). The dashboard compatibility endpoints [/predict](#), [/heatmap](#), and [/auth/*](#) are intentionally available at the root.

Authentication

Login

[POST /auth/login](#) or [POST /api/v1/auth/login](#)

```
{
  "username": "lea_user",
  "password": "lea_pass"
}
```

Demo accounts:

| Role | Username | Password | Frontend view |

| --- | --- | --- | --- |

| LEA | [lea_user](#) | [lea_pass](#) | Tactical heatmap and alerts |

| Admin | [admin_user](#) | [admin_pass](#) | Analytics, simulation, reports, administration |

Response:

```
{
  "access_token": "JWT_TOKEN",
  "token_type": "bearer",
  "role": "lea",
  "expires_in": 28800
}
```

Send the token on protected requests:

Authorization: Bearer JWT_TOKEN

The frontend should store the token for the session, decode the returned role only for routing, and treat the API response as authoritative for permission failures. Handle 401 by returning to login and 403 with a permission message.

Product Flow

1. User logs in.
2. Frontend loads `/predict` and `/heatmap` for the map and ranked ATM list.
3. Frontend loads `/api/v1/alerts` for the alert feed.
4. Selecting an ATM loads its risk, history, and explanation.
5. Selecting an alert allows an authorized user to acknowledge, escalate, resolve, or notify.
6. Admin users can view velocity and model metrics, inject a demo spike, and export a CSV report.

Core Frontend Features

1. ATM risk heatmap

GET `/heatmap?state=Uttar Pradesh`

Returns GeoJSON:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Point", "coordinates": [80.9462, 26.8467] },
      "properties": {
        "atm_id": "ATM_00001",
        "risk_score": 0.86,
        "severity": "high",
        "district": "Lucknow",
        "state": "Uttar Pradesh",
        "bank_name": "HDFC Bank",
        "cross_state_flag": true,
        "stale": false
      }
    }
  ]
}
```

Map rules:

- `risk_score < 0.40`: low / green
- `0.40 <= risk_score <= 0.70`: medium / amber
- `risk_score > 0.70`: high / red
- Show a stale indicator when `stale` is `true`.
- Coordinates are `[longitude, latitude]`, as required by GeoJSON.
- Never treat a high score as proof of a crime; label it as predicted risk.

2. Ranked ATM predictions

GET /predict?state=Uttar Pradesh&limit=50&refresh=false

Response:

```
{
  "predictions": [
    {
      "atm_id": "ATM_000001",
      "lat": 26.8467,
      "lng": 80.9462,
      "risk_score": 0.86,
      "district": "Lucknow",
      "state": "Uttar Pradesh",
      "bank_name": "HDFC Bank",
      "cross_state_flag": true,
      "stale": false
    }
  ],
  "generated_at": "2026-09-20T10:00:00+00:00"
}
```

Use this for a sortable priority table. Sort descending by `risk_score`, display the score as a percentage, and show cross-state and stale badges.

3. ATM details and explanation

- GET /api/v1/atms?state=...&district=...&page=1&page_size=20
- GET /api/v1/atms/{atm_id}
- GET /api/v1/atms/{atm_id}/risk
- GET /api/v1/atms/{atm_id}/risk-history
- GET /api/v1/atms/{atm_id}/risk-explanation

The explanation response contains:

```
{
  "atm_id": "ATM_000001",
  "risk_score": 0.86,
  "factors": [],
  "cross_state": {},
  "spike": {}
}
```

The frontend should render factors as evidence behind the prediction: complaint density, complaint velocity, mule proximity, cross-state activity, and spike status. Avoid presenting the model as a final investigation decision.

4. Alert feed

List alerts:

GET /api/v1/alerts?state=...&district=...&severity=...&status=...

Compatibility feed:

GET /alerts/feed

Important alert fields:

- **severity**: WARNING or CRITICAL
- **complaint_count**: complaints in the detected window
- **rolling_avg**: comparison baseline
- **cross_state**: whether the alert involves cross-state mule activity
- **status**: NEW, ACKNOWLEDGED, IN_REVIEW, ESCALATED, RESOLVED, or DISMISSED
- **message**: human-readable reason

Alert actions:

- POST /api/v1/alerts/{alert_id}/acknowledge
- POST /api/v1/alerts/{alert_id}/escalate
- POST /api/v1/alerts/{alert_id}/resolve
- GET /api/v1/alerts/{alert_id}/deliveries
- POST /api/v1/alerts/{alert_id}/notify

The UI should update the alert status after an action and show a timestamped activity state. Do not silently retry state-changing actions.

5. Complaint velocity analytics

Admin only:

GET /analytics/velocity?days=7&states=Uttar Pradesh,Maharashtra

Response:

```
{
  "trend": [
    { "date": "2026-09-20", "state": "Uttar Pradesh", "complaint_count": 42 }
  ]
}
```

Use a line or area chart grouped by state. The API pre-populates zero-count dates, so the frontend can plot the response directly.

Model metrics:

GET /analytics/metrics

Displays model metadata such as ROC-AUC, Precision@10, training time, and feature columns when available.

6. Cross-state entity network

- GET /api/v1/mule-accounts/{mule_id}/network
- GET /api/v1/network/entities/{entity_type}/{entity_id}/network

Response:

```
{
  "nodes": [
```

```

    { "id": "MULE_00001", "type": "mule_account", "label": "MULE_00001" }
  ],
  "edges": [
    { "source": "MULE_00001", "target": "ATM_00001", "type": "linked_atm", "weight": 1.0 }
  ]
}

```

Render nodes and edges as an investigation view. Node and edge types are backend data, not fixed UI labels.

7. Simulation controls

Admin only:

- GET /simulation/status
- POST /simulation/inject-spike
- POST /simulation/mode

Inject request:

```
{ "stage": "all", "fast": true }
```

The scripted scenario creates:

1. A **CRITICAL** cross-state event: Rajasthan complaints linked to UP mule accounts.
2. A **WARNING** single-state velocity spike in UP.

After injection, refresh predictions and alerts. Show the operation as a controlled simulation, not as live evidence.

Mode request:

```
{ "mode": "scripted", "rate_per_second": 1.5 }
```

Allowed modes are **random** and **scripted**.

8. CSV intelligence export

Admin only:

GET /reports/export

The response is **text/csv** with a download filename of **predictions.csv**. The frontend should trigger a browser download and not attempt to parse it as JSON.

9. Complaints, transactions, investigations

Complaint endpoints:

- POST /api/v1/complaints
- GET /api/v1/complaints
- GET /api/v1/complaints/{complaint_id}
- PATCH /api/v1/complaints/{complaint_id}/status

Transaction endpoints:

- `POST /api/v1/transactions`
- `GET /api/v1/transactions`
- `GET /api/v1/transactions/{transaction_id}`
- `GET /api/v1/transactions/{transaction_id}/trail`

Investigation endpoints:

- `POST /api/v1/investigations`
- `GET /api/v1/investigations`
- `GET /api/v1/investigations/{investigation_id}`
- `POST /api/v1/investigations/{investigation_id}/events`

These are the drill-down workflow behind an alert. The frontend should show loading, empty, validation-error, unauthorized, and server-error states for each list or detail view.

Admin-only Areas

The following require an admin role:

- `/analytics/*`
- `/reports/export`
- `/simulation/*`
- `/api/v1/admin/*`
- manual alert trigger and notification operations

Admin endpoints:

- `GET /api/v1/admin/users`
- `PATCH /api/v1/admin/users/{user_id}`
- `POST /api/v1/admin/atms/import`
- `GET /api/v1/admin/statistics`
- `GET /api/v1/admin/audit-logs`

Error Handling

Expected status codes:

Status	Meaning	Frontend action
---	---	---
200	Successful read or action	Update the view
201	Created	Add returned object to the list or navigate to detail
400	Invalid business input	Show field or operation error
401	Missing or invalid token	Clear session and show login

| 403 | Authenticated but not allowed | Show permission message |

| 404 | Resource not found | Show empty/not-found state |

| 409 | Conflicting operation, such as active simulation | Disable duplicate action and explain why |

| 422 | Request validation failed | Display validation details |

| 500 | Unexpected server failure | Show retry state and log correlation information if available |

Frontend Environment

For local development:

```
VITE_API_BASE_URL=http://localhost:8000
```

Use one API client with:

- a shared base URL,
- JSON headers by default,
- bearer-token injection,
- centralized 401 handling,
- response parsing for CSV downloads,
- request cancellation for map filters and search.

Recommended Frontend Screens

1. Login: username/password, demo account shortcuts, invalid-login state.
2. LEA dashboard: risk map, top ATM table, alert feed, selected ATM drawer.
3. Alert detail: severity, reason, evidence factors, status actions, delivery status.
4. Admin dashboard: velocity chart, model metrics, simulation controls, network view.
5. Complaint/investigation detail: complaint facts, transaction trail, related mule accounts, investigation events.
6. Report export: button with progress and downloaded CSV confirmation.

Acceptance Checklist

- ☐ Login works for LEA and Admin.
- ☐ Protected requests include the bearer token.
- ☐ Unauthorized users cannot access protected data.
- ☐ LEA cannot access admin-only screens/actions.
- ☐ Map renders GeoJSON coordinates in the correct order.
- ☐ Risk scores and stale state are visually clear.
- ☐ Alert severity and cross-state context are visible.
- ☐ Alert actions update status and handle failures.
- ☐ Admin simulation refreshes the map and alert feed.
- ☐ Analytics chart handles zero-count dates.

- [] CSV export downloads instead of rendering JSON.
- [] Loading, empty, validation, 401, 403, 404, 409, and 500 states are designed.
- [] No frontend action describes a prediction as a confirmed crime.